# SA-MIRI Course Schedule (7/11 Tentative)

| Practical Session | acronim | Book Chapter included |
|---|---|---|
| Ph | OPTIMIZE PT | task 11.1 – Find the maximum viable batch size<br>task 11.2 – Investigating dataloader bottleneck with a lightweight model<br>task 11.3 – Optimizing dataloaders with appropriate num_workers<br>task 11.4 – Confirming dataloader efficiency with vit<br>task 11.5 – Enable mixed precision with vit + micro-224<br>task 11.6 – Reproducing the effect of torch.compile()<br>task 11.7 – Report your conclusions<br>task 11.8 – Minor code tweaks for a final throughput boost |
| Pi | SCALING PT | task 12.1 – Reproducing distributed training results on mn5<br>task 12.2 – Analyze and compare scaling efficiency<br>task 12.3 – Investigate diminishing returns in training time<br>task 12.4 – Find the sweet spot for your use case |
| ~~Pj~~ | END-TO-END HF | ~~task 14.1 – Obtain your hugging face access token~~<br>~~task 14.2 – Download and run the hel~~<br>~~task 14.3 – Download the model and~~<br>~~task 14.4 – Transfer the model and da~~<br>~~task 14.5 – Run the inference and fine-tuning script on mn5~~<br>~~task 14.6 – Compare execution in colab vs mn5~~ |
| Pk | OPT & SCAL LLM | task 15.1 – Baseline experiment using facebook/opt-1.3b<br>task 15.2 – Finding the out-of-memory limit<br>task 15.3 – Mixed precision training<br>task 15.4 – Model precision<br>task 15.5 – Increasing batch size with model precision<br>task 15.6 – Enabling flash attention<br>task 15.7 – Increasing batch size with flash attention<br>task 15.8 – Using the liger kernel<br>task 15.9 – Augmenting batch size due to liger kernels<br>task 15.10 – Scaling on multiple gpus<br>task 15.11 – Final reflections |
| Pl | FUTURE | task 16.1 – Mapping the pendulum shifts of the triad<br>task 16.2 – Exploring the new giants of compute<br>task 16.3 – Powering the future of ai<br>task 16.4 – Charting the role of quantum in future supercomputing<br>task 16.5 – From data farms to ai preserves<br>task 16.6 – Embodiment as a source of learning |

This assignment has been withdrawn due to Google Colab resource limitations, as it cannot be completed with the free tier.

# SA-MIRI Course Schedule (7/11 Tentative)

| day | Wednesday | day | Friday |
|---|---|---|---|
| 12/11 | 11. Practical Guide to Efficient Training with PyTorch (Ph)<br>Ph<br>Presentation Pf | 14/11 | Ph |
| 19/11 | 12. Parallelizing Model Training with Distributed Data Parallel (Pi)<br>Ph/Pi<br>Presentation Pg | 21/11 | Pi |
| 26/11 | Honoris Causa Dr. Oriol Vinyals<br>(no class) | 28/11 | Pi |
| 03/12 | 13. Introduction to Large Language Models<br>14. End-to-End Large Language Models Workflow<br>Presentation Ph | 05/12 | Pk |
| 10/12 | 15. Exploring Optimization and Scaling of LLMs<br>Presentation Pi | 12/12 | Pk* |
| 17/12 | Presentation Pl (16. Looking Forward: HPC and AI Futures)<br>All groups will present | 19/12 | Pk*<br>“attendance not required (for students returning home)” |

* Pk No presentation, submission only.

# PI

| PI - FUTURE | GROUP A | GROUP B |
|---|---|---|
| TASK 16.1 — MAPPING THE PENDULUM SHIFTS OF THE TRIAD | | |
| TASK 16.2 — EXPLORING THE NEW GIANTS OF COMPUTE | | |
| TASK 16.3 — POWERING THE FUTURE OF AI | | |
| TASK 16.4 — CHARTING THE ROLE OF QUANTUM IN FUTURE SUPERCOMPUTING | | |
| TASK 16.5 — FROM DATA FARMS TO AI PRESERVES | | |
| TASK 16.6 — EMBODIMENT AS A SOURCE OF LEARNING | | |
| TASK 16.7 — The European AI Act and the Quest for Computational Sovereignty | | |
| TASK 16.8 — Bringing Inference to the Edge | | |

# PI

- **Task 16.1 — Mapping the Pendulum Shifts of the Triad**
- **Task 16.2 — Exploring the New Giants of Compute**
- **Task 16.3 — Powering the Future of AI**
- **Task 16.4 — Charting the Role of Quantum in Future Supercomputing**
- **Task 16.5 — From Data Farms to AI Preserves**
- **Task 16.6 — Embodiment as a Source of Learning**

# Task 16.7

- **TASK 16.7 — The European AI Act and the Quest for Computational Sovereignty**

  The European Union has taken a global lead in regulating artificial intelligence through the development of the AI Act, a legislative framework that seeks to ensure the safe, ethical, and trustworthy use of AI systems. This regulation introduces a risk-based approach and places responsibilities on developers, deployers, and users of AI.

  In this task, your objective is to explain the key principles and mechanisms of the EU AI Act. Can be focus on:

  - The motivations behind this regulation.
  - Its structure and classification of risk levels.
  - The obligations it imposes on different actors in the AI ecosystem.
  - Its implications for developers, especially in academic or open-source contexts.
  - The broader concept of digital and computational sovereignty in Europe: why does it matter who controls the compute and where models are hosted?

# Task 16.8

- **TASK 16.8 — Bringing Inference to the Edge**

  While training large AI models still requires massive computing infrastructure, inference (i.e., using those trained models) does not necessarily have to happen in centralized cloud environments.

  In this task, you are asked to explain how and why inference is moving away from the cloud and toward the edge — that is, to personal devices, local servers, and distributed micro-infrastructures. Your explanation could cover:

  - The technical and economic motivations for this shift (e.g., latency, privacy, energy efficiency, connectivity).
  - Examples of devices or scenarios where running AI locally makes sense.
  - The limitations and trade-offs involved (e.g., model size, accuracy, hardware constraints).
  - Emerging technologies and research directions enabling this shift (e.g., model quantization, distillation, hardware acceleration).

# Lab Pf presentation

- 1 group **will be randomly chosen**
  - We'll sum 4 numbers from randomly chosen students and use the '%' function with the total number of students to find the winner in the list.

```
>>> nums_to_add = ...+...+...+...
>>> winner= nums_to_add %
num_students +1
>>> print (winner)
```

| | |
|---|---|
| 1 | ALONSO GARCIA, MARTÍ; |
| 2 | ANDREU GERIQUE, DAVID; |
| 3 | ARNABOLDI, LEONARDO; |
| 4 | ARÓSTEGUI GARCÍA, ALBERTO; |
| 5 | AUBERTIER, THOMAS; |
| 6 | CRAMER, TIM; |
| 7 | DAHLBOM, IRMA; |
| 8 | DÍAZ CALDERÓN, MARC; |
| 9 | ESCOLÀ CURCÓ, POL; |
| 10 | HENLE, PATRICK; |
| 11 | HÖPFL, LEON; |
| 12 | KARG, CHISTIAN; |
| 13 | KOCH ENES, ANTONIO MANUEL; |
| 14 | KOLEVA, ELIZABET; |
| 15 | LÓPEZ CASTILLÓN, PAU; |
| 16 | MENA DOZ, VÍCTOR; |
| 17 | MOLDOVEANU, ANDREI; |
| 18 | OLIVERA LOYOLA, JUAN JOSE; |
| 19 | PAXTON, LAURA; |
| 20 | RABADAN ARROYO, RAÜL; |
| 21 | RIOS LÓPEZ, ADRIÁN; |
| 22 | RITTER, MAXIMILIAN FALK; |
| 23 | SELIGA, JAKUB; |
| 24 | SIMON, WILLIAM; |
| 25 | SOLDERA GIRELLI, VALÉRIA; |
| 26 | WALSH, ERIC; |

# 11. Practical Guide to Efficient Training with PyTorch

**Supercomputing for Artificial Intelligence**

**Foundations, Architectures, and Scaling Deep Learning Workloads**

Jordi TORRES.AI

# Book:

**11.2 Illustrative Case Study**

Custom Datasets
Models: ViT and Custom
Code Walkthrough: The train.py Script

**11.3 Setting up for Experiment Execution**

Python code outputs
SLURM Job Submission Scripts

**11.4 Tuning the Maximum Batch Size Hyperparameter**

Motivation: Why Does Batch Size Matter?
Methodology: Empirical Search
Experimental Results

**11.5 Efficient Data Handling: Preventing DataLoader Bottleneck**

Observing the Bottleneck with a Lightweight Model
Increasing num_workers to Unlock Performance
Returning to ViT: When DataLoader Parallelism Is Hidden

**11.6 Mixed Precision and Tensor Cores**

Understanding Floating-Point Formats
Mixed Precision Training with AMP
Experimental Results

**11.7 Torch Compile**

# Content

- **This chapter explores optimization techniques for improving resource utilization during model training.**
  - Focus on *hands-on experimentation* using PyTorch on a supercomputing platform.
  - Optimization requires *iterative experimentation*, not fixed recipes
  - Techniques covered include:
    - Hyperparameter tuning
    - Efficient data handling
    - Mixed precision training
    - JIT compilation

# By the end of this topic, students will be able to:

- Understand and tune key performance factors in PyTorch training.
- Identify bottlenecks (compute, memory, I/O).
- Apply mixed precision to exploit Tensor Cores.
- Use PyTorch’s compile features for speedups.
- Run reproducible performance experiments on a supercomputer.
- Ready to start next topic

# Illustrative Case Study

# Illustrative Case Study

- **Model of Study: Vision Transformer (ViT)**
  - ViT represents modern transformer-based architectures dominating deep learning (PyTorch implementation).
  - Provides a realistic context for:
    - Studying memory behavior
    - Evaluating DataLoader efficiency
    - Testing mixed precision and JIT compilation

# Illustrative Case Study

- **Dataset: ImageNet**
  - ImageNet: **14M images**, 20K categories → canonical large-scale CV dataset.
- **Custom Datasets for Controlled Experiments**
  - Training the full dataset is impractical in an academic setting.
  - Created two datasets derived from ImageNet.
    - All images resized to 224 × 224.
    - Enable realistic training experiments with manageable runtime and resource requirements.
    - Maintain structural complexity of real image classification tasks.
  - We use these datasets to:
    - Retain meaningful training behavior
    - Keep runtimes reasonable
    - Allow students to test multiple optimization techniques

# Illustrative Case Study

- **Why Tiny-ImageNet Works Well Here**
  - Not intended for final-accuracy benchmarking.
  - Designed to *stress the compute and I/O pipeline enough* to observe performance effects.
  - Enables rapid testing of:
    - Batch size scaling
    - Worker parallelism
    - Precision changes
    - JIT compilation
    - Overall training throughput

# Datasets

- **tiny-224 Dataset**
  - ~100,000 images (500 samples × 200 classes).
  - Large enough to show meaningful training behavior.
  - Small enough for academic HPC environments.
  - Used as the *main dataset* in most experiments.
- **micro-224 Dataset**
  - ~1/8 the size of tiny-224.
  - Enables fast turnaround while keeping classification structure intact.
- **Dataset Folder Structure**

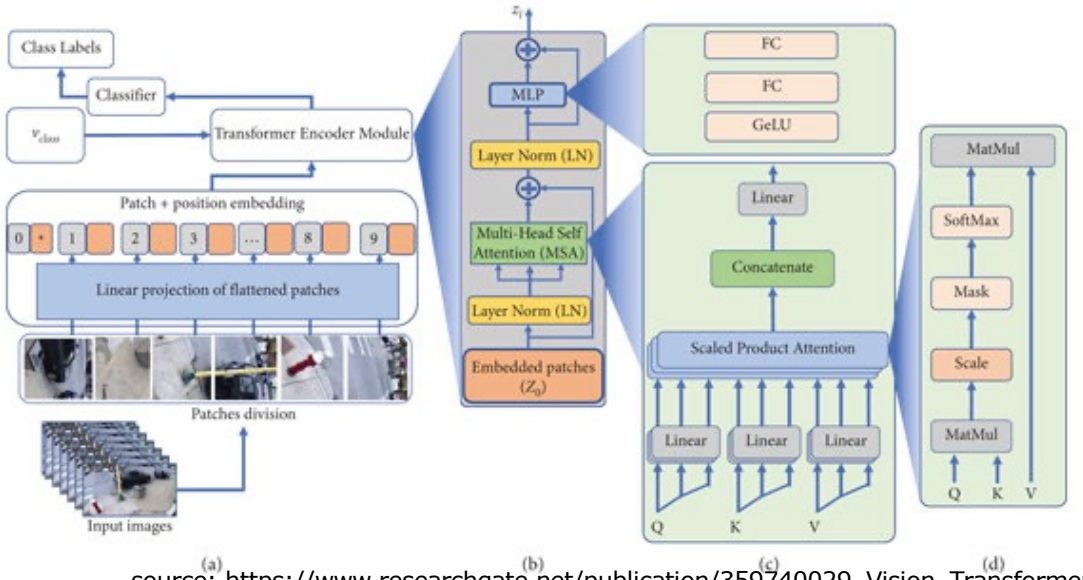
```
└── datasets
    ├── micro-224
    │   ├── train
    │   └── val
    └── tiny-224
        ├── train
        └── val
```

# Dual-Model Strategy

- **We work with two complementary models to support different experimental goals:**
- **ViT-H/14**
  - Large transformer model (~600M parameters).
  - Suitable for studying:
    - Mixed precision
    - JIT compile
    - Batch-size scaling
    - GPU memory behavior
- **Custom Lightweight Model**
  - ·Fully connected MLP (~400M parameters).
  - 4× faster per epoch than ViT.
  - Ideal for isolating Dataloader, optimizer, and throughput issues.

# Vision Transformer (ViT-H/14)

- Splits images into 14 × 14 patches.
- Treats patches as tokens in a transformer encoder.
- Represents modern large-scale architectures used in LLMs.
- Loaded from Hugging Face via transformers library.
- Excellent model for exploring PyTorch optimizations.




source: https://www.researchgate.net/publication/359740029_Vision_Transformer_and_Deep_Sequence_Learning_for_Human_Activity_Recognition_in_Surveillance_Videos

# Custom MLP Model (MyCustomModel)

- **Simple multilayer perceptron:**
  - Flatten → Linear → ReLU → … → Output
- **Configurable intermediate dimensions.**
- **Purpose-built for:**
  - Pipeline benchmarking
  - Dataloader studies
  - Rapid experimentation
- **Provides ~0.40 s/epoch vs 1.63 s for ViT in serial mode.**

# The `custom_model.py` Implementation

- Implemented as a PyTorch nn.Module.
- Sequential list of fully connected layers.
- No convolutions or attention → much faster compute.

```
from typing import List
import torch.nn as nn

class MyCustomModel(nn.Module):
    def __init__(
        self,
        n_classes: int = 200,
        resolution: int = 64,
        intermidiate_dimensions: List[int] = [8192, 4096, 2048, 1024, 512, 256, 128, 64],
    ):
        super().__init__()
        self.n_classes = n_classes
        self.resolution = resolution
        self.intermidiate_dimensions = intermidiate_dimensions
```

# The `custom_model.py` Implementation

```
        # Build model
        layer_dims = [resolution * resolution] + intermidiate_dimensions + [n_classes]
        layers = []
        for input_dim, output_dim in zip(layer_dims[:-1], layer_dims[1:]):
            layers.append(nn.Linear(in_features=input_dim, out_features=output_dim))
            if output_dim != n_classes:
                layers.append(nn.ReLU())

        self.model = nn.ModuleList(layers)

    def forward(self, x):
        for layer in self.model:
            x = layer(x)
        return x
```

– Clean template that students can extend or modify.

# The `train.py` Script

- **Core training script used in all tasks.**
  **It allows experimentation with:**
  - Model type (custom or ViT-H/14)
  - Batch size
  - Precision mode: fp32 / fp16 / bf16
  - Optimizer
  - torch.compile JIT mode
  - Modular design → easy to extend and adapt.

# train.py: Imports and Configuration

```
import os
import time
import torch
from sklearn.metrics import accuracy_score
from torch import autocast
from torch.nn import CrossEntropyLoss
from torch.optim import SGD, Adam, AdamW
from torch.utils.data import DataLoader
from torchvision.models import resnet, vision_transformer
```

```
def main(args):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    use_amp = True if args.mixed_precision else False
```

# `train.py`: Dataset + DataLoader

- **Uses folder-based datasets (DatasetFolder).**
- **Two DataLoaders: train and validation.**
- **Controls:**
  - batch_size
  - num_workers
  - shuffling
- **Essential for studying input-pipeline efficiency.**
- **Key part of this chapter's optimization focus.**

```
train_ds = DatasetFolder(...)
valid_ds = DatasetFolder(...)

train_dl = DataLoader(...)
valid_dl = DataLoader(...)
```

# `train.py`: Model Instantiation

- **Select model via CLI argument:**
  `--model_name custom` or `--model_name vit_h_14`

  - ViT loaded from `torchvision.models.vision_transformer`
  - Custom MLP directly instantiated
  - Model moved to GPU automatically

```
if args.model_name == "custom":
    model = MyCustomModel(...)
else:
    model = vision_transformer.vit_h_14(...)
```

# `train.py`: JIT Compilation & Setup

- Model is moved to the selected devide (usually GPU)
- If compilation is requested, torch.compile() is used to enable JIT optimization, potentially accelerating execution.
- The optimizer is instantiated based on user selection, and the loss function used is cross-entropy, which is appropriate for multi-class classification problems

```
model.to(device)
if args.compile:
    model = torch.compile(model, mode="reduce-overhead")
optimizer = ...
criterion = CrossEntropyLoss()
```

# `train.py`: Training Loop

- **Standard training structure:**
  - Forward pass
  - Loss computation
  - (Optional) Mixed-precision with torch.autocast
  - Backward pass
  - Optimizer update
  - Logging progress
- **Designed to measure runtime, not accuracy.**

```
for epoch in range(1, args.num_epochs + 1):
    model.train()
    for iteration, (inputs, labels) in enumerate(train_dl):
        ...
```

# `train.py`: Validation Loop

- Runs every `epochs_eval` epochs
- Uses `torch.no_grad()` for efficiency
- Computes accuracy via scikit-learn
- Minimal but functional evaluation for tracking behavior

```
if epoch % args.epochs_eval == 0 or epoch == args.num_epochs:
    model.eval()
    with torch.no_grad():
        ...
```

# `train.py`: Final Reporting

- **Reports:**
  - Total training time
  - Throughput (images/sec)
  - GPU memory usage
- **These metrics are used across all experiments in Chapter 11 and 12.**

```
full_training_time = time.time() - ft0
report = report_memory()
log("Training finished!")
log(f"Training throughput: ...")
log(report)
```

# Supporting Modules

- **The training script is complemented with two support modules:**
- **utils.py**
  - Logging utilities
  - Argument parsing
  - Memory reporting
- **dataset.py**
  - Custom `DatasetFolder`
  - Two collators: (*)
    - MyCustomCollator (grayscale, MLP)
    - RGBCollator (RGB, ViT/ResNet)
  - Handles resizing, tensor conversion, batching

*(*) A collator (collate_fn) builds each batch inside the DataLoader: it merges samples, converts them into tensors, applies preprocessing, and ensures the batch matches the input format required by the model — effectively acting as the batch factory.*

# File Setup on MareNostrum 5

- **Datasets location:** `/gpfs/projects/nct_345`

```
$ tree -L 2 datasets
datasets
|-- micro-224
|   |-- train
|   `-- val
`-- tiny-224
    |-- test
    |-- train
    `-- val
```

- **Home directory: Code from GitHub** (*)

```
|-- Chapter.11.slurm-files
|   |-- Ex00.BenchmarkModels.slurm
|   |-- Ex01.MaxBatchSize.slurm
|   |-- Ex02.NumWorkers.slurm
|   |-- Ex03.OptDataLoaders.slurm
|   |-- Ex04.OptDataLoadersViT.slurm
|   |-- Ex05a.DataTypes.fp32.slurm
|   |-- Ex05b.DataTypes.fp16.slurm
|   |-- Ex05c.DataTypes.bf16.slurm
|   |-- Ex06.JIT.slurm
|   `-- Ex07.train.optimizado.slurm
|-- Chapter.12.slurm-files
|   |-- ddp_job_tiny_1n_1g.slurm
|   |-- ddp_job_tiny_1n_2g.slurm
|   |-- ddp_job_tiny_1n_3g.slurm
|   |-- ddp_job_tiny_1n_4g.slurm
|   |-- ddp_job_tiny_2n_4g.slurm
|   |-- ddp_job_tiny_4n_4g.slurm
|   `-- ddp_job_tiny_8n_4g.slurm
|-- benchmark_models.py
|-- custom_model.py
|-- dataset.py
|-- train.py
|-- train_ddp.py
`-- utils.py
```

(*)

```
cd <dir>
git clone https://github.com/jorditorresBCN/HPC4AIbook.git
cd HPC4AIbook
ls
```

# Setting up for Experiment Execution

# Setting up for Experiment Execution

- **Goal**

  • Establish a consistent workflow to run all experiments through SLURM job scripts (ensure that hyperparameters can be controlled entirely from the batch file)
  • Keep the Python code untouched, focusing solely on system-level performance

- **Key Idea**

  The *SLURM script is the interface*. Hyperparameters, dataset paths, model choice and precision modes are all set there, not in the Python file.

# Example Output (Excerpt)

```
Current hostname: as01r3b19
INFO - ############# Config #############
INFO - ### mixed_precision: None
INFO - ### batch_size: 32
INFO - ### eval_batch_size: 32
INFO - ### num_workers: 0
INFO - ### num_epochs: 5
INFO - ### epochs_eval: 5
INFO - ### iteration_logging: 500
INFO - ### model_name: vit
INFO - ### intermidiate_dimensions: None
INFO - ### pretrained: False
INFO - ### compile: False
INFO - ### optimizer: adamw
INFO - ### learning_rate: 0.0005
INFO - ### dataset: /gpfs/<path>/datasets/micro-224
INFO - ### resolution: 224
INFO - ### Device: cuda
INFO - ### AMP Enabled: False
INFO - ### Total number of parameters 631021000
INFO - ### Total training samples (batches): 12480 (390)
INFO - ### Total validation samples (batches): 9984 (312)
INFO - ###################################
INFO - [EPOCH: 1] Starting training loop...
INFO - [EPOCH: 1] Training throughput: 78.153 imgs/s
          .
          .
          .
INFO - [EPOCH: 5] Accuracy: 0.031
INFO - Training finished!
INFO - Complete training Time: 826 s
INFO - Training throughput:    344 imgs/s
INFO - GPU memory reserved:    28 GB
```

# Python Code Outputs

- **The training script prints structured information:**
  - Hyperparameter configuration (batch size, epochs, num_workers…)
  - Model details (name, param. count, pretrained / AMP / compile)
  - Dataset info (path, number of samples, batches)
  - Per-epoch throughput
  - Final metrics:
    - Complete training time (wall-clock)
    - Avg training throughput (imgs/s)
    - GPU memory reserved
- **Purpose**
  - Verify SLURM configuration
  - Track throughput and memory usage
  - Support reproducible experiments

*Note: The first epoch is excluded from performance analysis due to initialization overhead.*

# SLURM Job Submission Scripts

- **Job Directives**

```
#SBATCH --job-name NumWorkers
#SBATCH --chdir .
#SBATCH --output ./results/R-%x.%j.out
#SBATCH --error ./results/R-%x.%j.err
```

- **Resource Allocation**

```
#SBATCH --nodes 1
#SBATCH --ntasks-per-node 1
#SBATCH --gres gpu:1
#SBATCH --cpus-per-task 20
#SBATCH --time 02:00:00
#SBATCH --account <account>
#SBATCH --qos acc_debug
#SBATCH --exclusive
```

# SLURM Job Submission Scripts

- **Environment Setup**

```
module purge
module load singularity
```

- **Experiment Configuration Variables**

```
MODEL=custom              # --model_name: custom | vit

DS=./datasets/tiny-224    # --dataset: tiny-224 | micro-224
EPOCHS=5                  # --num_epochs
BS=10                     # --batch_size & --eval_batch_size
NW=0                      # --num_workers
OPTIM=sgd                 # --optimizer
LOG_ITER=500              # --iteration_logging
EPOCHS_EVAL_FREQ=5        # --epochs_eval
```

# SLURM Job Submission Scripts

- **Command Assembly**

```
PYTHON_FILE=./train.py
PYTHON_ARGS="--model_name $MODEL \
             --dataset $DS \
             --num_epochs $EPOCHS \
             --batch_size $BS \
             --eval_batch_size $BS \
             --num_workers $NW \
             --optimizer $OPTIM \
             --iteration_logging $LOG_ITER \
             --epochs_eval $EPOCHS_EVAL_FREQ \
            "
export CMD="python3 $PYTHON_FILE $PYTHON_ARGS"
```

Key Idea: The entire experiment configuration becomes a single command string executed inside the container

# SLURM Job Submission Scripts

- **Container Configuration**

```
SINGULARITY_CONTAINER=/gpfs/apps/MN5/ACC/SINGULARITY/SRC/images/nvidiaPytorch24.07
SINGULARITY_ARGS=" --nv  $SINGULARITY_CONTAINER"
```

`--nv` passes NVIDIA drivers + GPUs into the container

The container includes PyTorch, CUDA, cuDNN, NCCL

Ensures consistent results across runs and nodes

# SLURM Job Submission Scripts

- **Job Launch with srun and Singularity**

```
SRUN_ARGS="--cpus-per-task $SLURM_CPUS_PER_TASK --jobid $SLURM_JOB_ID"
srun $SRUN_ARGS bsc_singularity exec  $SINGULARITY_ARGS bash -c "$CMD"
```

  - Final flow:
    - `srun` allocates resources
    - `bsc_singularity` enters the container
    - `bash -c "$CMD"` runs training inside the container

# Tuning the Maximum Batch Size Hyperparameter

# Why Batch Size Matters

- **One of the most influential hyperparameters in deep learning training. Affects:**
  - Training throughput (images/s)
  - GPU memory usage
  - Convergence and update frequency
- **Larger batches:**
  - Keep GPU compute units busy for longer
  - Reduce number of optimizer steps per epoch
- **Limitation:**
  - Memory scales with batch size
  - Exceeding GPU capacity triggers CUDA Out-Of-Memory (OOM)
- **Goal:** find the maximum viable batch size, i.e., the largest that fits in memory

# Methodology: Empirical Search

- **Hyperparameter tuning is often empirical**
- **Conduct multiple training runs**
- **Double the batch size each time:**
  **16 → 32 → 64 → 128**
- **Use:**
  - **ViT-H/14** (high memory demand)
  - **micro-224** dataset (fast execution, realistic patterns)
- **Adjust only the batch size in the SLURM script**
  - Stop when the job fails with CUDA OOM
  - Typical error excerpt:

```
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 482.00 MiB. GPU 0 has a total
capacity of 63.43 GiB of which 117.50 MiB is free. Including non-PyTorch memory, this process
has 63.30 GiB memory in use. Of the allocated memory 62.35 GiB is allocated by PyTorch, and
298.30 MiB is reserved by PyTorch but unallocated.
```

# Experimental Results

| Batch Size (img) | Complete Training Time(s) | Throughput (img/s) | Memory (GB) |
|---|---|---|---|
| **16** | 901 | 79 | 14 |
| **32** | 764 | 84 | 24 |
| **64** | 731 | 88 | 46 |
| **128** | - | - | OOM |

Table 11.1 – Maximum batch size test results using the ViT model and micro-224 dataset.

# Experimental Results

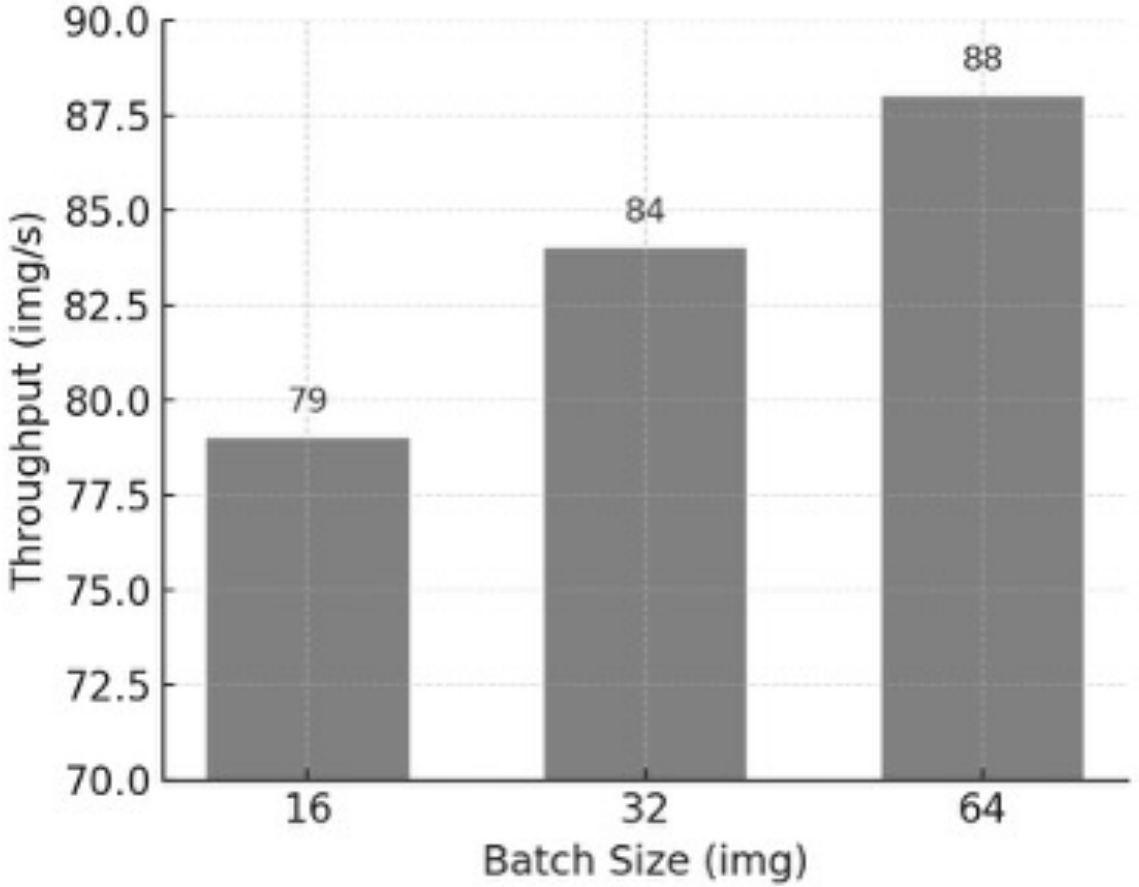



Figure 11.1 – Training throughput vs batch size for the ViT model on micro-224.

# Experimental Results

- **Interpretation of Results**
  - Throughput increases significantly from 16 → 64
  - Gains between 32 → 64 are smaller
  - Batch size 128 fails due to insufficient GPU memory
  - Memory usage scales almost linearly with batch size
  - Upper bound on an H100 with ViT-H/14 = Batch Size 64

*This experiment models real-world hyperparameter tuning under hardware constraints*

# Task 11.1

- **Task 11.1 – Find the Maximum Viable Batch Size**
  - Reproduce the experiment described in this section by training the ViT model on the micro-224 dataset using different batch sizes. Launch the train.py script via SLURM, adjusting only the batch size in each run. Identify the maximum batch size that completes training successfully without triggering an out-of-memory error.
  - Once you have completed the tests, summarize your results in a table following the format of Table 11.1. Are your measurements equivalent to those presented here?. Reflect on how this empirical approach helps you understand the memory constraints and compute efficiency of large-scale training.

# Efficient Data Handling: Preventing DataLoader Bottleneck

# The Often-Ignored Bottleneck: Data Loading

- **In HPC training with PyTorch, the bottleneck is not always the GPU**
- **The PyTorch DataLoader:**
  - Retrieves samples from disk
  - Applies transforms
  - Assembles batches
  - Optionally loads data **in parallel** via num_workers

- **If data preparation is too slow:**
  **GPU waits → throughput collapses → poor resource utilization**

# Why num_workers Matters

- **Each worker = a separate CPU process handling:**
  - JPEG decoding
  - Resizing / transforms
  - Tensor conversion

- **Increasing `num_workers` parallelizes these tasks**

- **Goal: keep GPU fed with batches**
  - If not: GPU idles, even if the model is tiny and batch size is large

# Experiment Setup: Exposing the Bottleneck

- **Switch to lightweight custom model (`MODEL=custom`)**
  - Small compute cost
  - Makes data pipeline the dominant factor
- **Use micro-224 dataset**
  - Smallest dataset (1/8 of tiny-224)
  - Fast to run, ideal for observing pipeline behavior
- **Vary batch size: 32 → 64 → 128 → 256**
- **Since the model is tiny, GPU memory never becomes the limit**
- **This isolates the effect of data loading performance**

# Experimental Results

| Batch Size (img) | Complete Training Time(s) | Throughput (img/s) | Memory (GB) |
|---|---|---|---|
| **32** | 1204 | 1546 | 3 |
| **64** | 289 | 1765 | 3 |
| **128** | 270 | 1864 | 3 |
| **256** | 269 | 1863 | 3 |

Table 11.2 – Experiment 2 results using the custom model and the micro-224 dataset: training time, throughput, and memory usage for different batch sizes.

# Experimental Results

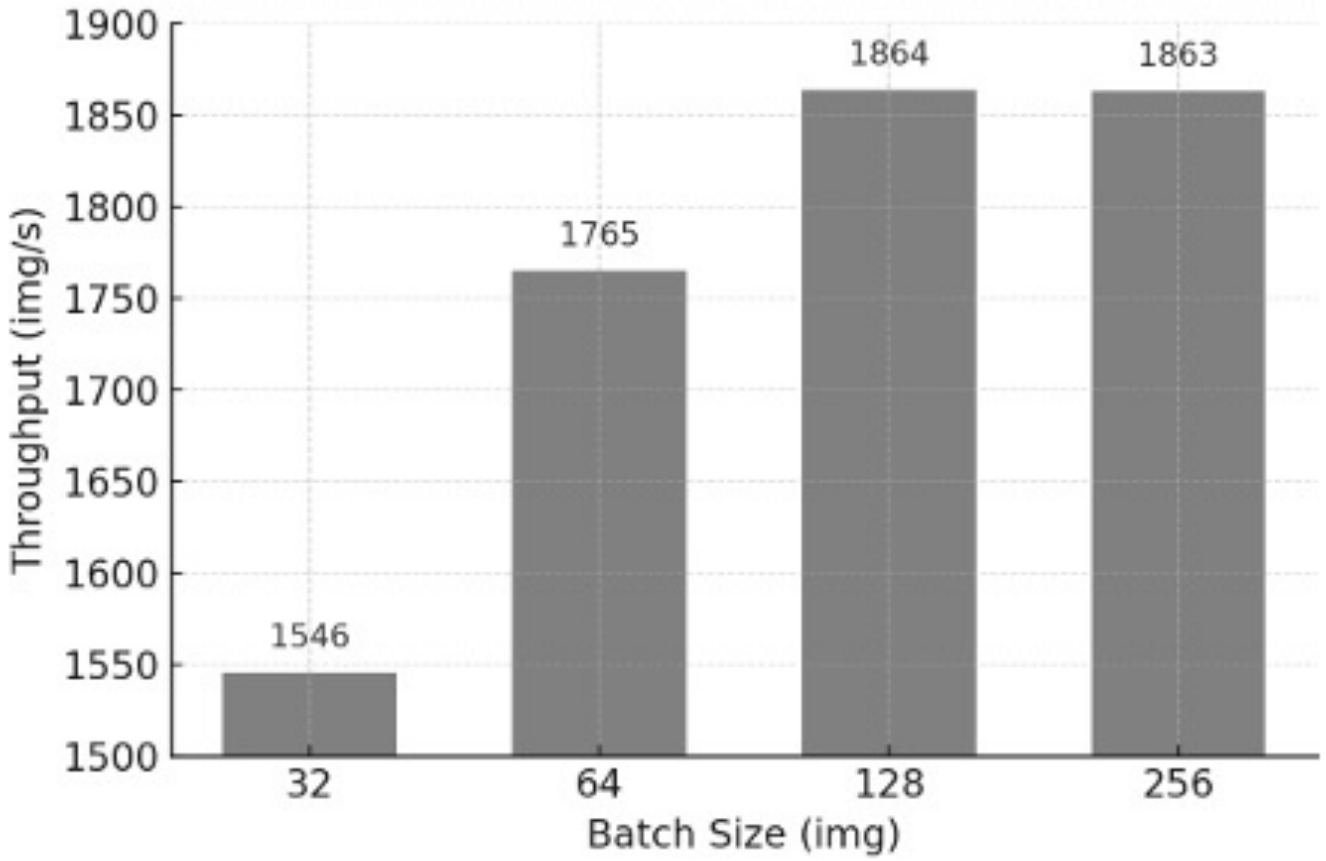



Figure 11.2 – Throughput performance across increasing batch sizes using the custom model on the micro-224 dataset.

# Interpreting the Results

- Memory usage remains flat (≈3 GB) — the model is small
- Throughput improves from 32 → 64 → 128, then plateaus
- Increasing batch size beyond 128 does not improve throughput
- This indicates:
  - GPU is no longer the bottleneck
  - CPU-side data handling cannot keep up

# Diagnosis: GPU Underutilization

- **Key Takeaways**
  - Faster models make DataLoader bottlenecks more visible
  - Increasing batch size does not always improve throughput
  - Training performance is a balance of:
    - GPU compute
    - CPU preprocessing
    - Disk I/O
    - Parallel data loading
  - **Fix = increase `num_workers`**

# Task 11.2

- **Task 11.2 – Investigating the DataLoader Bottleneck with a Lightweight Model**
  - Reproduce the experiment described in this section using the custom model (MODEL=custom) and the micro-224 dataset. Perform several training runs with increasing batch sizes (up to 256) and record the complete training time, throughput, and memory usage for each configuration.

  - Once your table of results is complete, compare them with Table 11.2. Are your observations consistent? Pay special attention to GPU utilization—use nvidia-smi during training to check how busy the GPU remains. Can you confirm that GPU idling occurs as batch size increases? What does this tell you about the role of the DataLoader and CPU-side preprocessing in the overall training pipeline?

# Why num_workers Matters

- **Goal: Fixing the DataLoader bottleneck.**
  - In the previous experiment, throughput saturated even with large batch sizes.
  - The GPU remained *idle*, waiting for batches prepared by the CPU.
  - Cause: DataLoader running with num_workers = 0 (everything done in the main process).
  - Large batches + fast model = GPU starves for data.
  - Solution: parallel data loading using num_workers.

# What num_workers Does

- **num_workers = number of subprocesses loading + preprocessing data in parallel.**
- **Each worker reads images, decodes JPEGs, applies transforms.**
- **More workers = more batches prepared simultaneously.**

- **Critical when:**
  - Using lightweight models.
  - Using image datasets with non-trivial preprocessing.
  - Training on fast GPUs (H100 = extremely sensitive to slow feeders).

# Experiment Setup

- **Experiment**: Fix batch size = 256
  Vary num_workers ∈ {0, 1, 2, 4, 8, 10, 16, 20}

- Model: custom (very lightweight).
- Dataset: micro-224.
- Where to change it: In DataLoader or SLURM script.
- Focus metrics:
  - Training time
  - Throughput
  - Memory usage
  - GPU utilization

# Table of results

| Number of Workers | Complete Training Time(s) | Throughput (img/s) | Memory (GB) |
|---|---|---|---|
| **0** | 1192 | 1598 | 3 |
| **1** | 237 | 2163 | 3 |
| **2** | 128 | 3871 | 3 |
| **4** | 72 | 6953 | 3 |
| **8** | 47 | 10818 | 3 |
| **10** | 46 | 10881 | 3 |
| **16** | 47 | 10746 | 3 |
| **20** | 48 | 10496 | 3 |

Table 11.3 – Impact of increasing num_workers on training throughput (batch size 256, custom model, micro-224 dataset)

- **Observation: >5× speedup compared to workers = 0.**

# Graphical results

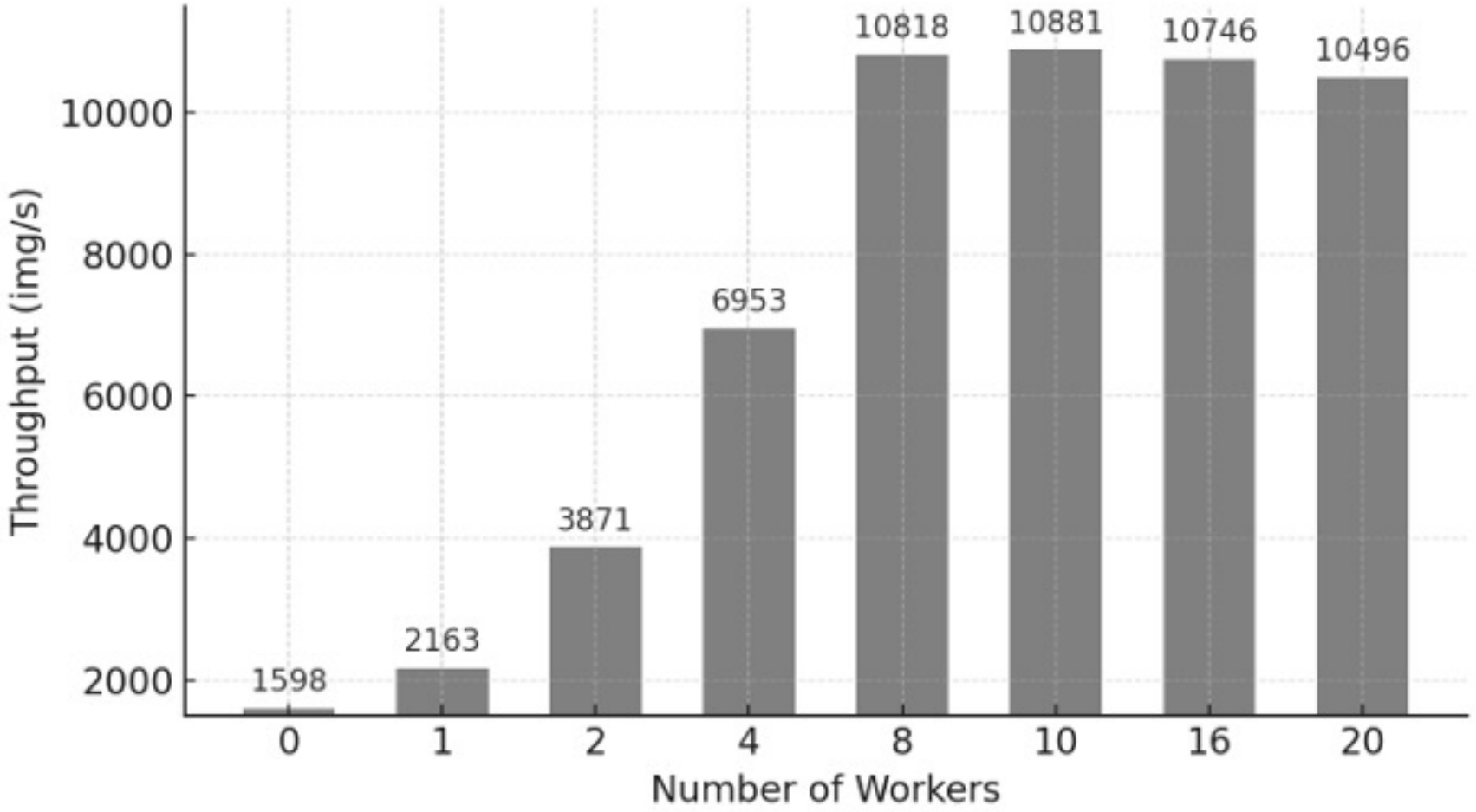



Figure 11.3 – Training throughput as a function of num_workers. Substantial gains are observed up to 8 workers.

# Throughput vs num_workers

- **Key Shape:**
  - Slow beginning
  - Exponential → linear growth
  - Plateau at ~8–10 workers
  - Diminishing returns afterwards
- **Interpretation:**
  - From 0 to 8 workers, CPU becomes less of a bottleneck.
  - Beyond ~10 workers, GPU no longer waits for data.
  - Any further increase gives minimal or no gains.

# Task 11.3

- **Task 11.3 – Optimizing DataLoaders with appropriate `num_workers`**
  - In this task, you will explore the performance impact of the num_workers parameter in PyTorch's DataLoader using the custom model introduced in the previous section. Your goal is to empirically determine how parallelizing data loading affects the overall training throughput and to identify the optimal number of workers for your specific setup.
  - Start by using the same setup as in the previous experiment: the custom model (MODEL=custom), the micro-224 dataset, and a batch size of 256. Repeat the training multiple times, each time modifying the value of num_workers in the SLURM script (or in the DataLoader configuration). Suggested values are: 0, 1, 2, 4, 8, 10, 16, and 20.
  - Record your observations, focusing on total training time, throughput, and GPU memory usage. Do the results match those presented earlier? How does throughput evolve as more workers are added? Is there a point at which increasing num_workers no longer helps?
  - The objective of this task is to deepen your understanding of how data pipeline configuration influences training performance, especially in compute-bound vs I/O-bound situations. This is a critical skill when working with large datasets and high performance systems.
  - Document your findings in a table and plot, and briefly explain the observed behavior. What would be your recommendation for setting num_workers in future experiments?

# Returning to ViT: When DataLoader Parallelism Is Hidden

- **Returning to the ViT model**
  We switch back from the lightweight custom network to the Vision Transformer (ViT), which will act as our baseline for the next tasks.

- **Why return to ViT?**
  The ViT is heavy enough in compute that it lets us observe the opposite behaviour:
  - GPU compute dominates
  - DataLoader latency becomes almost invisible
  - num_workers barely affects throughput

# Experimental setup

**We evaluate the impact of num_workers when training ViT:**

- **Configuration**
  - Model: *vit*
  - Dataset: *micro-224*
  - Batch size: 64
  - num_workers: 0, 1, 2, 4, 8, 10, 16, 20
- **The goal:**
  Determine whether the DataLoader still limits performance with a compute-intensive model.

# Results overview

| Number of Workers | Complete Training Time(s) | Throughput (img/s) | Memory (GB) |
|---|---|---|---|
| **0** | 824 | 88 | 46 |
| **1** | 701 | 95 | 46 |
| **2** | 701 | 92 | 46 |
| **4** | 701 | 91 | 46 |
| **8** | 702 | 91 | 46 |
| **10** | 702 | 91 | 46 |
| **16** | 702 | 91 | 46 |
| **20** | 703 | 91 | 46 |

Table 11.4 – Impact of num_workers on training performance for the ViT model (batch size 64, micro-224 dataset).

# Why num_workers barely matters

- **With ViT, compute time per batch is large.**

- **While the GPU processes batch *n*,**
  **the CPU loads batch *n+1* in parallel, so I/O gets hidden.**

- **This creates the ideal scenario:**
  - GPU fully saturated
  - DataLoader “just fast enough”
  - Adding more workers brings no benefit
  - No visible bottleneck

# The takeaway

- **Lightweight models (custom CNN):**
  - very fast compute
  - easy to starve
  - DataLoader is the bottleneck
  - tuning num_workers is essential
- **Heavy models (ViT):**
  - slow per batch
  - GPU always busy
  - data loading overlaps compute
  - tuning num_workers has minimal impact
- **The DataLoader bottleneck depends on *model compute intensity*, not only on dataset.**

# The takeaway

- **To ensure consistency and avoid surprises across experiments:**
  - We set num_workers = 10 as the default value (works well for ViT and avoids starving lighter models)

# Task 11.4

- **Task 11.4 – Confirming DataLoader Efficiency with ViT**
  - Repeat the experiment described above using the ViT model and the micro-224 dataset, keeping the batch size fixed at 64. Adjust the number of workers in the DataLoader to the following values: 0, 1, 2, 4, 8, 10, 16, and 20.
  - Record the complete training time, throughput, and memory usage for each configuration, and summarize your findings in a table and chart. Do your results match those from Table 11.4? What differences do you observe compared to Task 11.3 with the custom model?
  - Use these observations to explain why the ViT model does not suffer from the same bottlenecks and how its compute characteristics interact with the data pipeline. This exercise will help you identify when tuning num_workers is critical and when it has negligible impact on performance.

# Mixed Precision and Tensor Cores

# Why Mixed Precision?

- **Goal:**
  - Accelerate training by changing the numerical format used in tensors.

    *Modern GPUs (like H100) include Tensor Cores, designed to operate with 16-bit precision much more efficiently.*

- **Benefits:**
  - Faster training
  - Reduced memory footprint
  - Enables larger batch sizes
  - Often minimal accuracy loss

# Floating-Point Formats in Deep Learning

- **PyTorch supports multiple floating-point types, but three dominate GPU training:**
  - fp32 (32 bits)
    - High precision
    - Safe but expensive (slow + high VRAM use)
  - fp16 (16 bits)
    - Very fast, tiny memory footprint
    - Reduced precision + dynamic range
    - May destabilize some models
  - bf16 (16 bits)
    - Same 8-bit exponent as fp32 (large dynamic range)
    - Fewer fraction bits
    - Fast and numerically stable
    - Ideal for modern GPUs

# Visual Comparison

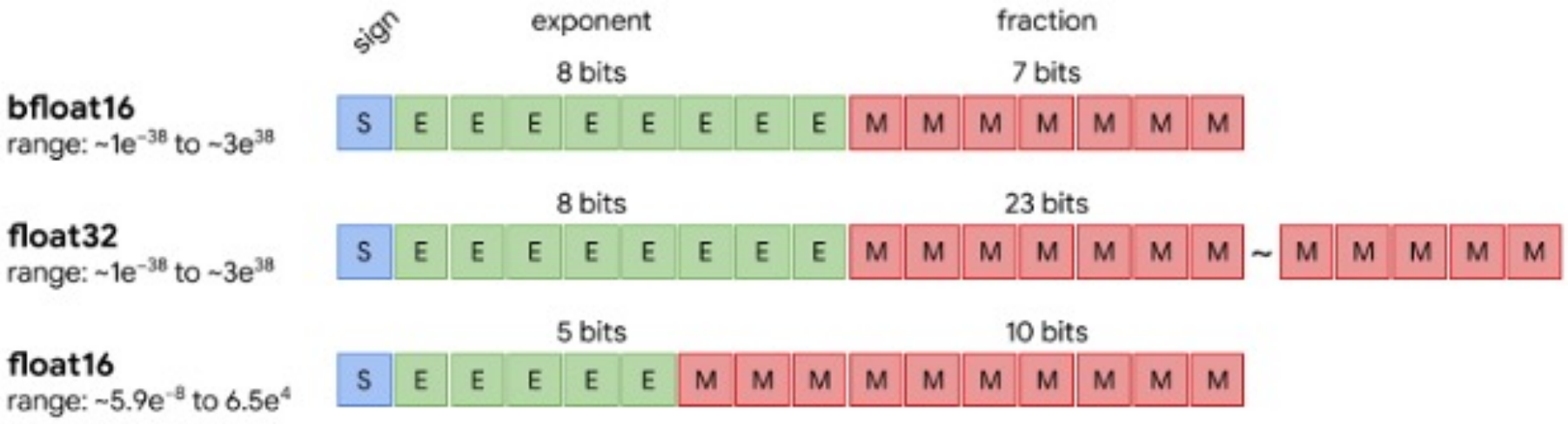



Figure 11.4 – Visual comparison of bfloat16, float32, and float16. While fp16 sacrifices both exponent and fraction precision, bfloat16 keeps the 8-bit exponent (like fp32) and reduces only the mantissa, preserving a similar dynamic range with reduced precision (Image source: image source: https://www.pragmatic.ml/a-survey-of-methods-for-model-compression-in-nlp/)

# What Is Mixed Precision Training?

- **Mixed Precision→blending precisions automatically**
  - Use low-precision (fp16/bf16) for heavy linear algebra operations
  - Use high precision for sensitive steps (loss scaling, some accumulations)
- **PyTorch makes this easy with `torch.autocast`**

- **With *AMP* (Automatic Mixed Precision) zero code changes to the model itself.**

# Why It Works: Tensor Cores

- **Tensor Cores accelerate:**
  - matrix multiplications
  - convolutions
  - attention blocks

- **Low-precision ops → up to 2–4× faster**

- **Memory reductions → fit larger batches into VRAM**

# Experimental Setup

- **Model: ViT**
- **Dataset: micro-224**
- **Batch sizes tested: 64 and 128**
- **Precisions: fp32, fp16, bf16**
- **AMP enabled through CLI flag:**
  - `--mixed_precision fp16`
  - `--mixed_precision bf16`

# Experimental Results

| Precision | Batch Size (img) | Throughput (img/s) | Memory (GB) |
|---|---|---|---|
| **fp32** | **64** | 92 | 46 |
| **fp32** | **128** | - | OOM |
| **fp16** | **64** | 182 | 27 |
| **fp16** | **128** | 190 | 50 |
| **bf16** | **64** | 186 | 27 |
| **bf16** | **128** | 196 | 50 |

Table 11.5 – Comparing different precision modes on ViT + micro-224.

# Key Observations

- **Switching from fp32 → bf16:**
  - ~2 × throughput
  - ~½ memory usage
  - Enables doubling batch size (64 → 128)

- **bf16 slightly faster than fp16:**
  - H100 optimizations
  - No need for dynamic loss scaling
  - Better stability

- **fp32 fails at batch size 128 due to memory constraints.**

# Practical Recommendation

- **For the rest of the experiments we will use:**
  - Precision: bf16
  - Batch size: 128
  - num_workers: 10

- **This combo provides:**
  - Consistent performance
  - Good numerical stability
  - Realistic HPC best-practice configuration

# Task 11.5

- **Task 11.5 – Enable Mixed Precision with ViT + micro-224**
  - Update your training script to use mixed precision with torch.autocast, and rerun the ViT model on micro-224. Try with both --mixed_precision=fp16 and --mixed_precision=bf16, (in slurm scripts) and measure the achieved throughput and memory usage.

# Torch Compile

# Why Compile?

- **torch.compile() (PyTorch 2.0+) enables:**
  - transforming the model into an optimized representation
  - graph-level operator fusion
  - reduced Python dispatch overhead
  - improved memory reuse
  - speedups without rewriting the model

- **Result:**

  faster training and inference with a single flag.

# How It Works (Intuition)

## Just-In-Time compilation:

- **First iteration**

  PyTorch analyzes the model and builds an optimized graph representation.

- **Subsequent iterations**

  The backend executes fused kernels with lower overhead and better memory locality.

- **Transparent to the user**

  No changes to model structure, dataloaders or training logic.

- **To enable compilation, simply pass:**

  --compile

# Experimental Setup

- **Model: ViT**
- **Dataset: micro-224**
- **Configuration:**
  - batch size = 128
  - precision = bf16
  - num_workers = 10
  - identical hyperparameters as previous sections
- **We compare:**
  - Without JIT
  - With JIT enabled

# Results

| JIT | Precision | Batch Size (img) | Throughput (img/s) |
|---|---|---|---|
| No | bf16 | 128 | 172 |
| Yes | bf16 | 128 | 265 |

Table 11.6 – Performance of ViT + micro-224 with and without `torch.compile()`. Compilation nearly doubles the throughput at batch size 128 using bf16 precision.

# Task 11.6

- **Task 11.6 – Reproducing the Effect of torch.compile()**
  - Run a version of the ViT training script with the --compile argument enabled, as described in this section. Compare the throughput to the baseline (without compilation) and confirm that you observe a value close to the one reported in Table 11.6.

# Task 11.7

- **Task 11.7 – Report your Conclusions**
  - Write a short summary (½ page max) of your own experimental results:
  - Table: list throughput and memory after each optimisation you applied.
  - Plot: show throughput versus experiment number for a quick visual of progress.
  - Reflection: identify which step delivered the largest improvement on your run and explain why, in one or two sentences.
  - Use this task to cement the habit of *evidence-based optimisation*: change one thing, measure, document—then move on.

# Ph: Practical Guide to Efficient Training with PyTorch

- **Tasks included:**
  - task 11.1 to task 11.7

- **Deliverable & Evaluation:**
  - Upload a single PDF (per group) to the racó@FIB intranet, containing **as many slides as you need per task to clearly express what is requested in each one**.

- **In class (evaluation day):**
  - One group (chosen at random) will give a clear, concise, and straight-to-the-point presentation.
    However, **do not worry about timing — it does not need to follow a strict "elevator pitch" style** as in previous labs.
    The goal here is pedagogical, allowing time to discuss results and reflect on what has been learned.